

INT428 Project Report (Project-Based Assessment)

Domain-Specific Generative AI Chatbot Using APIs

Student Name:

Roll Number:

Branch & Semester: Computer Science Engineering 4th Semester

Project Title: DevOps Copilot — Domain-Specific Generative AI Chatbot for DevOps Guidance

Faculty Name: Dr. Dimple Nagpal

Submission Date : 04-05-2026

1. Introduction

This project, **DevOps Copilot**, is a **domain-specific Generative AI chatbot** designed to assist users with DevOps-focused tasks such as generating Dockerfiles, suggesting test cases, identifying production readiness issues, and offering CI/CD guidance.

Unlike a general chatbot, this system uses **multi-agent orchestration (LangGraph)** to route user queries to specialized agents based on intent, and it **analyzes a local codebase** (via AST-based scanning) to provide accurate, context-aware DevOps outputs.

2. Assessment Objective

- **Meaningful real-world problem identification:** addresses repetitive DevOps tasks and time-consuming repo audits.
 - **Use of Generative AI APIs:** uses **Google Gemini** via API for LLM responses (as per repository documentation).
 - **Technical competence in an API-based AI system:** FastAPI backend + SSE streaming + agent routing + code analysis tools.
 - **Communication of design decisions:** system prompts, routing logic, and model parameters (Temperature/Top-p) are documented and justified.
-

3. Problem Identification & Innovation (10 Marks)

3.1 Problem Statement

DevOps engineers frequently need to perform repetitive tasks such as:

- auditing repositories for deployment readiness,
- writing or optimizing Dockerfiles,
- identifying dependency bloat,
- preparing test case suggestions,
- and answering general CI/CD and deployment questions.

These tasks often require **manual context-gathering** (reading many files) and repeated boilerplate work.

3.2 Proposed Solution

A **DevOps-domain chatbot** that:

1. Accepts a user query and (optionally) a path to a local codebase
2. Automatically analyzes the codebase structure using AST and file scanning
3. Routes the query to the best specialist agent
4. Uses a Generative AI API (Gemini) to produce a scoped DevOps output

3.3 Innovation / Originality

Key innovations from this repository:

- **Multi-agent DevOps specialization** (not one generic prompt)
- **Automatic context injection** using `code_analyzer.py` to reduce “copy-pasting context”
- **Intent routing** via a router node that selects an appropriate agent
- **Streaming responses** (FastAPI + SSE) for better UX during analysis

3.4 Societal / Practical Impact

This chatbot improves productivity by reducing time spent on manual audits and boilerplate generation. It helps students and early-career engineers learn DevOps best practices faster and assists teams in identifying production risks earlier in the delivery pipeline.

4. Domain Definition

Selected Domain: DevOps / Software Delivery Lifecycle Automation

Target Users: DevOps engineers, developers, student teams building deployable projects

Typical Queries:

- “Generate a Dockerfile for this project.”
 - “Is this codebase production-ready? What risks do you see?”
 - “Suggest test cases for this backend.”
 - “How can I reduce bundle size / dependency bloat?”
 - “Explain CI/CD pipeline recommendations for this app.”
-

5. Tools & Technologies Used

5.1 Model / API Details

- **API Used:** Google Gemini API
- **Model Name:** [Fill exact model, e.g., gemini-1.5-flash / gemini-1.5-pro]
- **Model Version/Release:** [If available]

5.2 Technology Stack

Backend

- FastAPI (REST API + SSE streaming)
- LangGraph & LangChain (agent orchestration and workflow)
- Python AST analysis for code understanding

Frontend

- React + TypeScript
- Vite
- Tailwind CSS + DaisyUI

Hosting/Deployment

- Localhost
-

6. Data Collection & Domain Knowledge Preparation

No training was performed, but domain understanding was demonstrated through:

- DevOps best practice knowledge (containerization, CI/CD, production readiness)
- Repository-based domain prompts (stored as Markdown skill files in [backend/app/prompts/skills/](#))
- Curated DevOps workflow knowledge encoded into specialist agent roles

Information Sources Studied:

1. <https://docs.docker.com/>
2. <https://kubernetes.io/docs/home/>

How domain knowledge influenced prompt design:

- System instructions enforce “DevOps-only scope”
 - Agents produce structured outputs (Dockerfile, risk report, test cases)
 - Safety rules: highlight unknowns, avoid fake claims, recommend verification steps
-

7. System Design & Architecture (Technical Execution)

7.1 Chatbot Type

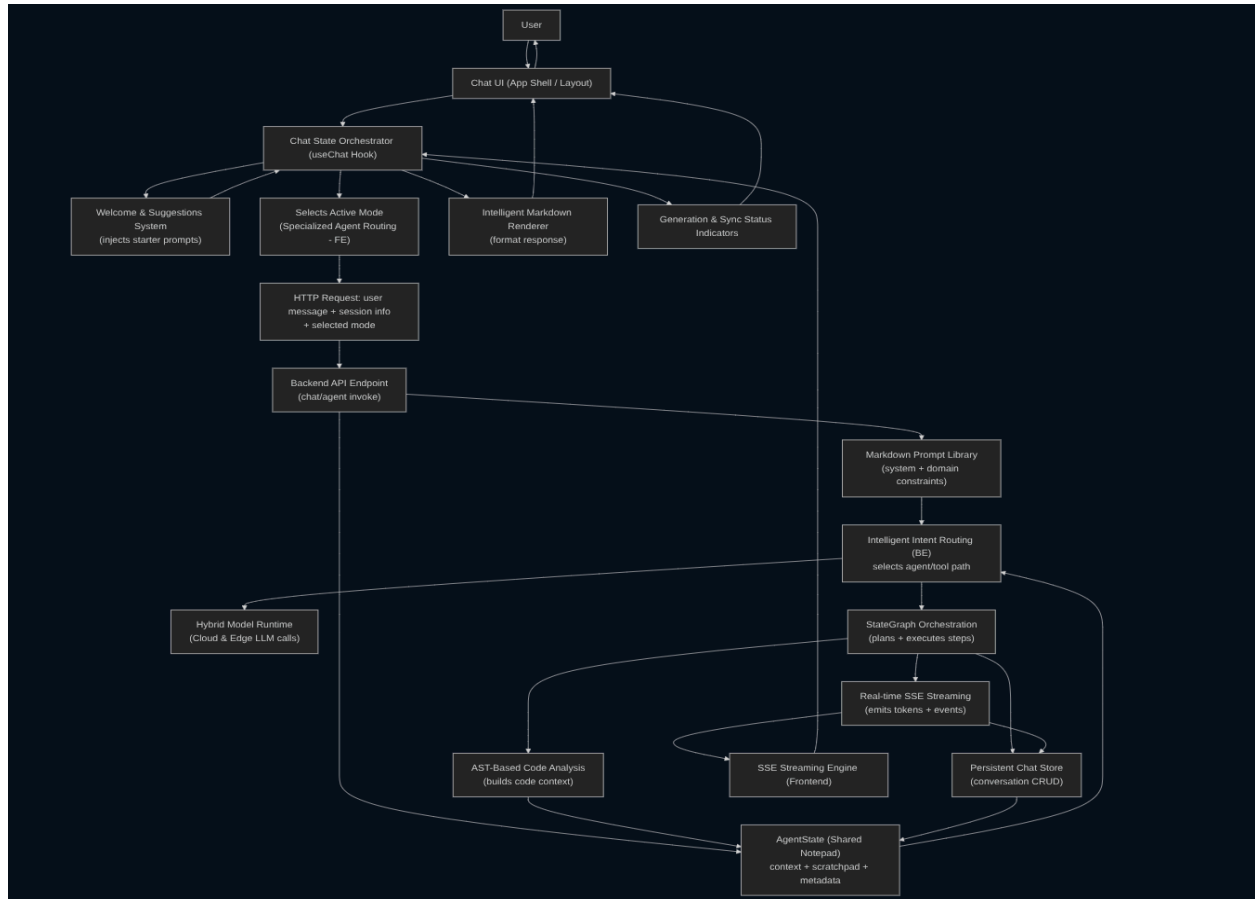
Type: Hybrid

- **Generative core (LLM) + tool-driven context retrieval (AST + file scanning) + agent routing**

7.2 Specialist Agents

- Dockerfile Agent
- Test Case Agent
- Bundle Size Agent
- Production Agent
- General Agent (DevOps-only guidance)

7.3 Data Flow



8. Prompt Engineering & Domain Control

8.1 Domain Constraints

Domain control is achieved using:

- Role-based agent prompts (DevOps specialist personas)
- Explicit instruction to remain in DevOps scope
- Structured response formatting (steps, risks, checklists, commands)

9. Model Configuration Awareness (Temperature & Top-p)

9.1 Parameters Used

Parameter	Value
Temperature	0.2–0.4
Top-p	0.5

9.2 Justification (DevOps Domain)

- **Lower Temperature** is preferred to produce consistent and factual, low-hallucination outputs (recommended for deployment instructions and risk reporting).
 - **Top-p** balances safety and completeness; moderate-to-high values help cover multiple relevant recommendations while staying constrained by system prompts.
-

10. Academic Integrity & Attribution

This project is original work. Tools and libraries used include:

- FastAPI
 - LangGraph / LangChain
 - Google Gemini API
 - React, TypeScript, Vite, TailwindCSS, DaisyUI
 - Python AST module
-

11. Conclusion

DevOps Copilot successfully demonstrates a domain-specific Generative AI chatbot that integrates a real Generative AI API and provides meaningful DevOps outputs through multi-agent routing, codebase analysis, and streaming responses. The project meets INT428 mandatory requirements and shows practical API integration, prompt engineering, and model configuration awareness.

Annexure A — Filled Evaluation Questionnaire (INT428)

Chatbot-Based Project Evaluation Questionnaire (INT428)

Student Name:

Roll Number:

Branch & Semester: Computer Science Engineering 4th Semester

Project Title: DevOps Copilot — Domain-Specific Generative AI Chatbot for DevOps Guidance

Faculty Name: Dr. Dimple Nagpal

Section A: Project Overview

Q1. Type of Chatbot Developed

- ☐ Rule-based
- ☐ Retrieval-based
- ☐ Generative (LLM-based)
- ☒ Hybrid

Q2. Platform Used for Deployment

(Select all that apply)

- ☒ Web Application
- ☐ Mobile Application
- ☐ Desktop Application
- ☐ Messaging Platform (WhatsApp/Telegram/Slack)
- ☐ Cloud API only (no UI)

Q3. Deployment Link / Access Details

Deployment URL / App Link: Localhost

Section B: Model & API Details

Q4. Type of API Used

- ☐ OpenAI API
- ☒ Google Gemini API
- ☐ Azure OpenAI API
- ☐ Custom REST API
- ☒ Local Model API

Q5. Model Name Used

Model Name: gemma-4-31b-it

Q6. Model Version

Model Version / Release: gemma-4-31b-it

Section C: Context & Data Handling

Q7. Contextual Memory Usage

- ☐ No memory (single-turn chatbot)
- ☐ Session-based memory
- ☒ Long-term memory (database/vector store)
- ☒ Hybrid memory approach

Q8. Flow of Data in the Chatbot

Briefly explain the data flow:

1. User sends query in the React chat UI
2. Frontend sends request to FastAPI backend
3. Backend uses router to detect intent and choose specialist agent
4. Backend optionally scans local codebase and generates structured context
5. Backend calls Gemini API with system prompt + context + user query
6. Backend streams response via SSE
7. Frontend displays streamed answer

Section D: Model Configuration & Behavior

Q9. Model Parameters Used

Parameter	Value
Temperature	0.2–0.4
Top-p	0.5

Q10. Thinking Level & Role Assignment

Thinking Level:

- ☐ Basic (direct answers)
- ☐ Intermediate (context-aware reasoning)
- ☒ Advanced (multi-step reasoning)

Role Assigned to Model:

- ☐ Assistant
- ☐ Tutor
- ☐ Customer Support Agent
- ☒ Domain Expert

Section E: Technology Stack

Q11. Technology Stack Used

(Mention tools & frameworks)

Frontend: React, TypeScript, Vite, TailwindCSS, DaisyUI

Backend: FastAPI , LangGraph / LangChain ,Python AST module

Database: Sqlite3

Hosting: Local

Section F: Implementation Evidence (Screenshots & Code)

Q12. API Call Screenshot

```
async def dockerfile_node(state: AgentState) -> dict:
    """Analyze the codebase and generate or optimise a Dockerfile."""
    runtime = state["model_runtime"]
    logger.info(
        "Starting Dockerfile agent for command=%s runtime=%s",
        state["command"],
        runtime,
    )
    llm = build_chat_model(runtime=runtime, temperature=0.3)
    system_prompt = get_prompt("dockerfile")

    user_msg = (
        f"User request: {state['command']}\n\n"
        f"### Codebase Analysis\n{state['code_context']}"
    )

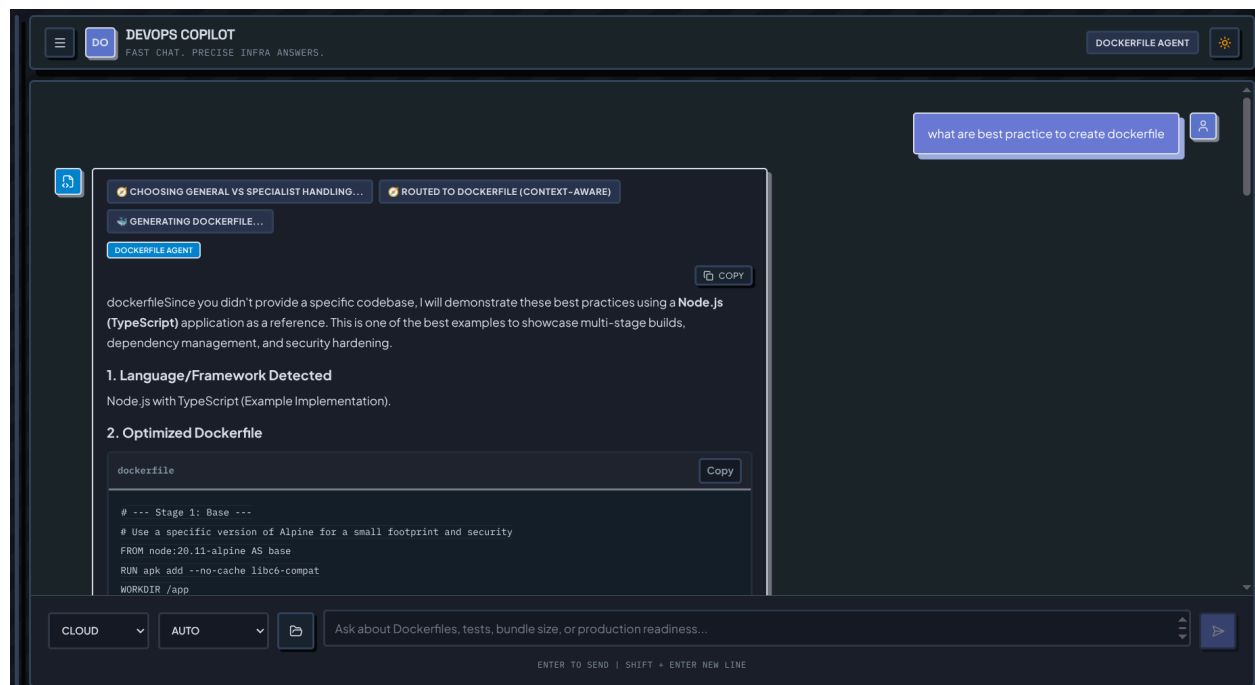
    logger.info("Invoking model for Dockerfile generation")
    response = await llm.ainvoke(
        [SystemMessage(content=system_prompt), HumanMessage(content=user_msg)]
    )
    logger.info(
        "Dockerfile agent completed; response length=%s",
        len(response.content) if response.content else 0,
    )
    return {"result": response.content, "messages": [response]}
```

```

2026-05-01 22:16:52,780 INFO devops_chatbot.agents.router_node: Routing command to intent: what are best practice to
create dockerfile
2026-05-01 22:16:52,781 INFO devops_chatbot.prompts.library: Loaded prompt 'routing-classifier' from /home/tanishq/Do
cuments/Development/Devops-chatbot/backend/app/prompts/skills/routing-classifier.md
2026-05-01 22:16:52,781 INFO devops_chatbot.prompts.library: Loaded prompt 'routing-context-aware' from /home/tanishq
/Documents/Development/Devops-chatbot/backend/app/prompts/skills/routing-context-aware.md
2026-05-01 22:16:52,782 INFO devops_chatbot.agents.router_node: Attempting context-aware routing using recent history
2026-05-01 22:16:56,142 INFO google_genai.models: AFC is enabled with max remote calls: 10.
2026-05-01 22:16:57,693 INFO httpx: HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemma-
4-31b-it:streamGenerateContent?alt=sse "HTTP/1.1 200 OK"
2026-05-01 22:16:59,588 INFO devops_chatbot.agents.router_node: Context-aware intent selected: dockerfile
2026-05-01 22:16:59,592 INFO devops_chatbot.api.routes: Routing selected: agent=dockerfile source=context-aware
2026-05-01 22:16:59,593 INFO devops_chatbot.api.routes: Progress step: 🌀 Generating Dockerfile...
2026-05-01 22:16:59,594 INFO devops_chatbot.agents.dockerfile_agent: Starting Dockerfile agent for command=what are b
est practice to create dockerfile runtime=cloud
2026-05-01 22:16:59,625 INFO devops_chatbot.prompts.library: Loaded prompt 'dockerfile' from /home/tanishq/Documents/
Development/Devops-chatbot/backend/app/prompts/skills/dockerfile.md
2026-05-01 22:16:59,625 INFO devops_chatbot.agents.dockerfile_agent: Invoking model for Dockerfile generation
2026-05-01 22:16:59,628 INFO google_genai.models: AFC is enabled with max remote calls: 10.
2026-05-01 22:17:01,378 INFO httpx: HTTP Request: POST https://generativelanguage.googleapis.com/v1beta/models/gemma-
4-31b-it:streamGenerateContent?alt=sse "HTTP/1.1 200 OK"
2026-05-01 22:17:40,600 INFO devops_chatbot.agents.dockerfile_agent: Dockerfile agent completed; response length=2
2026-05-01 22:17:40,603 INFO devops_chatbot.api.routes: Analysis complete; emitting result event agent=dockerfile sou
rce=context-aware files_analyzed=0
2026-05-01 22:17:40,736 INFO devops_chatbot.api.routes: Analysis streaming generator finished for path=
[0] 0:uv* 1:npm- "linux" 22:32 01-May-26

```

Q13. Chatbot Working Interface Screenshot



Q14. GitHub Repository Link

- Repository URL: <https://github.com/Tanishquppal220/Devops-chatbot>

Declaration

I confirm that the information provided above is accurate to the best of my knowledge.

Student Signature:

Date: 04-05-2026